Institución Universitaria	GUÍA DE TRABAJO Desarrollo de Software	Código	FDE 048
		Versión	03
		Fecha	2009-06-09

1. IDENTIFICACIÓN

dellоaristizabal@itm.edu.co

Asignatura	*Programación de Software*							Guía No.	3
Área	Básicas de la Tecnología					Nivel		4	
Código	580304006					Pensum		1	
Correquisito(s)	*580506004*					Prerrequisito(s)		NA	
Créditos	4	TPS	64	TIS	128	TPT		TIT	
TRABAJO INDEPENDIENTE						**TRABAJO PRESENCIAL**			
Trabajo Teórico	X	Trabajo Práctico	X			Trabajo Teórico		Trabajo Práctico	X

2. IDENTIFICACIÓN

COMPETENCIAS	CONTENIDO TEMÁTICO	INDICADOR DE LOGRO
Conceptos y componentes de la Programación Orientada a Objetos (POO).	Uso de IDE para la implementación de aplicaciones para entornos de escritorio y Web con lenguajes propios en Programación Orientada a Objetos (POO). Crear Instancias de clase y mensajes entre objetos para interacción de funcionabilidades de clase y acciones a realizar en la aplicación.	Construir una aplicación donde se demuestre la propiedad de herencia o derivación de clases en la programación orientada a objetos, a partir de la representación UML. Establece las características de la clase para emplear la herencia o derivación simple, el polimorfismo y uso de la sobrecarga. Construir una clase haciendo uso de las características de la POO (Encapsulación, Abstracción, polimorfismo y Herencia).

3. RECURSOS REQUERIDOS

API Web - Básica

Sala de sistemas con 40 equipos con Visual Studio 2026, 1 proyector de video y marcadores varios.
Material bibliográfico

4. PROCEDIMIENTO

4.1 Consultar

1. ¿Qué es una API?
2. ¿Qué es un servicio Web?
3. ¿Cuáles son los diferentes tipos de APIs?
4. ¿Cuáles son las ventajas y desventajas de una API como servicio?

4.2 Problema para solucionar

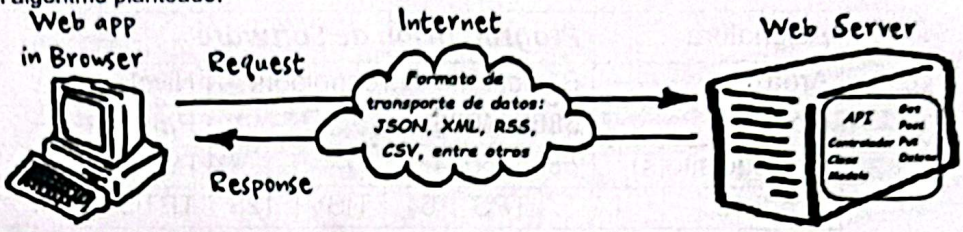
"SusCopias SAS" desea contratarlo para realizar un programa que permita calcular y dar como salida el pago del servicio realizado a un cliente por concepto de copias en blanco y negro (Carta(C), Oficio(O) y Extra-Oficio(E), teniendo en cuenta que se capturan los siguientes datos: Nombre y Nro de doc. del cliente y la *cantidad de copias por cada tipo de copia*. El programa debe pedir inicialmente el valor de cada copia y mostrarlos en cada tipo (Para no poderlos modificar). Mostrar en cada servicio el valor a pagar por cada tipo de copia, el subtotal, el porcentaje y total de descuento otorgado, valor IVA y total a pagar por el servicio. *Nota*: Si la cantidad *total* de copias en el servicio está entre 50 y 100 se otorga un descuento del 10% o si es mayor a 100 del 15% y el impuesto de IVA es del 7.5%. *No necesariamente se facturan todos los tipos de copias al mismo tiempo.*

Actividad 1. Crear un algoritmo para solucionar el problema planteado, donde se determinen las entradas, el proceso a emplear y las salidas requeridas. Realizar la respectiva prueba de escritorio. **(Traerlo realizado)**

Powered by CamScanner

Actividad 2. Establecer el diseño a construir con sus correspondientes componentes de entrada, salida y comportamientos, acorde al algoritmo planteado.

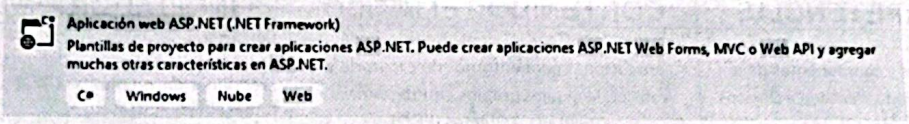
Nota: El cliente tiene la página **html** que va a utilizar el servicio que existe en la API ejecutada en el Web Server utilizando formato: **JSON**. El Web Server contiene la **API** que recibirá el requerimiento (request) del cliente en el enlace desde internet, el controlador del servicio API (bajo el método: Post), se realizara el proceso operativo en la clase bajo el procedimiento adecuado utilizando el modelo, y; retornará (response) la información requerida



Actividad 3. Aplicación API WEB para realizar el servicio Web API

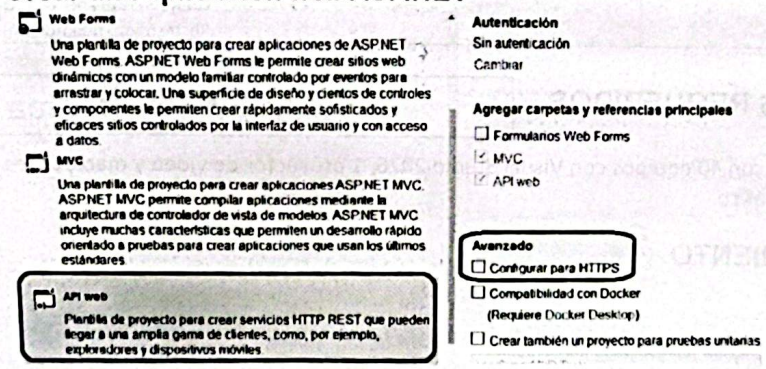
a. Crear la carpeta: **Practica3.**

b. En Microsoft Visual Studio 2026, utilizar la opción: ***Crear un Proyecto***, buscar la opción:

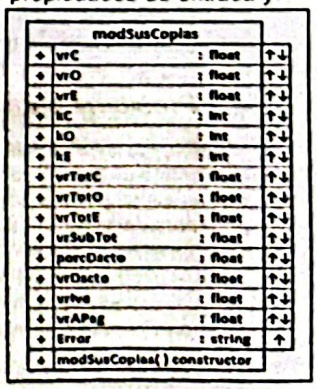


proyecto tipo: *Aplicación web ASP.NET (.NET Framework)* en C#, con nombre: ***apiSusCopias***, en la carpeta creada en la actividad anterior. *Aceptar.*

Te lleva a la ventana de para *crear una aplicación web ASP.NET*, en ella seleccionar: ***API web*** y ***deschequear*** la opción: ***Configurar para HTTPS***, luego: ***Crear***



c. En la carpeta **Models**, agregar una clase local con nombre: **modSusCopias** con las propiedades de entrada y salida como se muestra en el modelo UML para la clase.



c.1. Las propiedades son de entrada y salida; `vrC`, `vrO` y `vrE` (float) para los valores unitarios de cada tipo de copia (carta, oficio y extraoficio respectivamente), `kC`, `kO` y `kE` (int) para cantidad de cada tipo de copia (carta, oficio y extraoficio respectivamente), `vrTotC`, `vrTotO` y `vrTotE` (float) para los valores por cada tipo de copia (carta, oficio y extraoficio respectivamente), `vrSubTot` (float) para el subtotal de la factura, `porcDscto` (float) para el valor del porcentaje del descuento, `vrDscto` (float) para el valor del descuento otorgado, `vrIva` (float) para el valor a pagar por impuesto de IVA y `vrAPag` (float) para el valor total a pagar y `Error` (cadena) para contener el mensaje de error.

c.2. Crear el **constructor** de la clase, en cero los numéricos y vacío la cadena (Define el archivo Json de entrada y retorno).

Powered by CamScanner

d. Crear una nueva carpeta con nombre: **Clases.**

d.1. Agregar una clase local con nombre: **clsOpeSusCopias**, será la clase de operaciones del servicio.

d.2. Usar la carpeta: **Models** del proyecto para ser reconocida la clase: **modSusCopias.**

```
using apiSusCopias.Models;
```

d.3. Crear la única propiedad y los métodos privados y público acorde al siguiente modelo UML para la clase.

clsOpeSusCopias			
+	pMod	: modSusCopias	↑↓
-	Validar( )	: bool	
-	hallarDescuento( )	: void	
+	Facturar( )	: void	

d.3.1. Crear la propiedad: `pMod` desde la clase modelo: `modSusCopias` de entrada y salida.

```
public modSusCopias pMod { get; set; }
```

d.3.2. El método privado **Validar()** que valida la información de la propiedad: `pMod` que no sea vacío; y de todas las entradas: `vrC, vrO y vrE` (mayores a 0), `kC, kO y kE` (mayores o iguales a 0) y retornar el valor booleano *true* para decir correcto o *false* ante algún error.

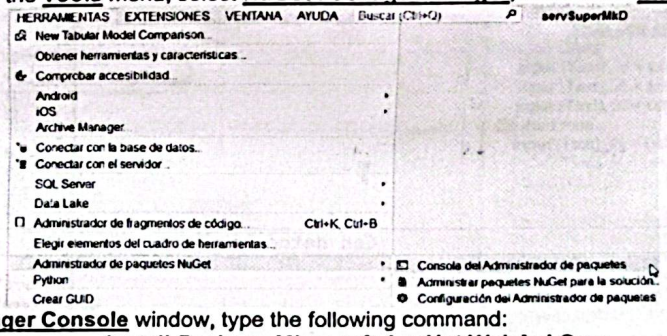
d.3.3. El método privado **hallarDescuento( )**, encuentra el valor del porcentaje y el valor total del descuento; a partir de la cantidad total de copias, no retorna ningún valor.

d.3.4. El método público **Facturar()**, invoca el método: **Validar**, si falla se sale, en caso contrario; halla el valor de cada tipo de copia y el subtotal, invoca el método **hallarDescuento** para los valores correspondientes al descuento y luego; halla el valor del IVA y el valor total a pagar.

e. Instalar y habilitar el componente **Cors.**

Cors es una característica de seguridad del navegador que restringe las solicitudes HTTP de origen cruzado que se inician desde secuencias de comandos que se ejecutan en el navegador. Ver: https://docs.microsoft.com/en-us/aspnet/web-api/overview/security/enabling-cross-origin-requests-in-web-api

e1. In Visual Studio, from the **Tools** menu, select **NuGet Package Manager**, then select **Package Manager Console**.



e2. In the **Package Manager Console** window, type the following command:

Install-Package Microsoft.AspNet.WebApi.Cors

e3. In file App_Start/**WebApiConfig.cs**: in the method **Register**, add instruction:

config.EnableCors();

f. En la carpeta **Controllers.**

f.1. Agregar una **clase de controlador web API (v2.1)** con nombre: **servSusCopiasController**, será la clase de control del servicio, tanto de seguridad con el componente: ***Cors***, como de los datos de entrada y de salida dado en las estructuras tipo ***JSon***, siendo el componente para la recepción y transmisión de datos a través de las URLs del cliente y el servidor. **NOTA**: *No quitar o reemplazar la palabra* `Controller`*, por cuanto es palabra clave para conocer el nombre del servicio de la API*

f.2. Usar las carpetas: **Models** y **Clases** del proyecto para ser reconocida la clase modelo y la clase de operación, además; usar la clase del componente **Http.Cors** instalado anteriormente

```
using apiSusCopias.Models;
using apiSusCopias.Clases;
using System.Web.Http.Cors;
```

f.3. Habilitar el componente **Cors** de manera manual para el enlace del cliente que va a utilizar el servicio de la API, esta habilitación se realiza por fuera (antes) de la clase controladora, la instrucción es la siguiente:

```
// Colocarla antes de la clase
// Adicionar manualmente -> Para habilitar el Cors y el servicio a consumir el cliente
        [EnableCors(origins: "http://localhost:XXXXX", headers: "*", methods: "*")]
            // XXXXX, es el número del puerto del enlace donde corre el form (Proyecto web)
```

f.4. En el método ***Post*** (único método que se va a utilizar en este proyecto como IO de información) hacer que retorne un objeto de tipo: ***modSusCopias*** y reciba como argumento o parámetro el valor ***objIN*** de tipo: ***modSusCopias*** (todos los datos de la clase modelo)

Codificar de la siguiente forma:

```
public modSusCopias Post([FromBody] modSusCopias objIN)  //Utilizando el modelo como I/O
{  clsOpeSusCopias objOpe = new clsOpeSusCopias( );
```


Powered by CamScanner

```
        objOpe.pMod = objIN; //Paso del parámetro de entrada a la propiedad de la clase: clsOpeSusCopias
        objOpe.Facturar( );
        return objOpe.pMod; //retorna con los datos de entrada y los de salida o error ocurrido
    }
```

g. probar el servicio con el software de pruebas de APIs: Insomnia o Postman.

En el enlace: https://insomnia.rest/download (Insomnia) o https://www.postman.com/downloads/ (Postman) descargar la versión de prueba y realizar prueba de funcionamiento del servicio creado en la API.

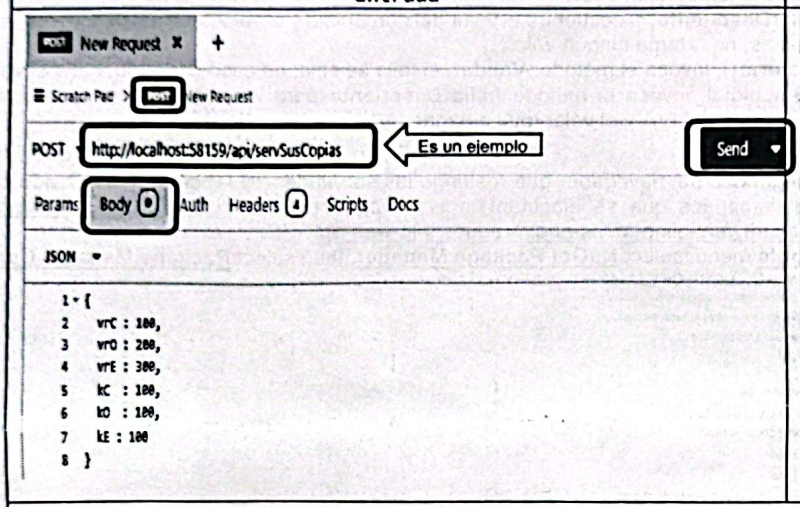
g.1. Ejecutar el proyecto de la Api y capturar el link desde donde está corriendo (observar el número del puerto) y complementarlo con **/api/servSusCopias**. Ver siguiente ejemplo (el número del puerto cambia para cada estudiante):

https://localhost:XXXXX/api/servSusCopias

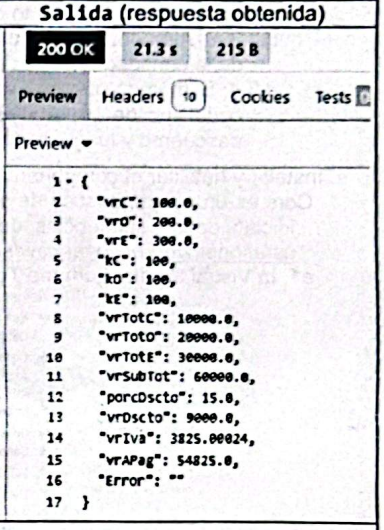
g.2. En **Insomnia** seleccionar: ***Post***, *ubicar link del servicio*, ***Body***, en la lista: **JSON**

g.3. Crear el texto tipo: JSON con los datos de entrada y presionar el botón: **Send**

Entrada



Salida (respuesta obtenida)



Con datos erróneos

```
{
  vrC : 100,
  vrO : 200,
  vrE : -300,
  kC  : 100,
  kO  : 100,
  kE : 100
}
```

```
{
  "vrC": 100.0,
  "vrO": 200.0,
  "vrE": -300.0,
  "kC": 100,
  "kO": 100,
  "kE": 100,
  "vrTotC": 0.0,
  "vrTotO": 0.0,
  "vrTotE": 0.0,
  "vrSubTot": 0.0,
  "porcDscto": 0.0,
  "vrDscto": 0.0,
  "vrIva": 0.0,
  "vrAPag": 0.0,
  "Error": "Vr. Copia ExtraOficio no válido"
}
```

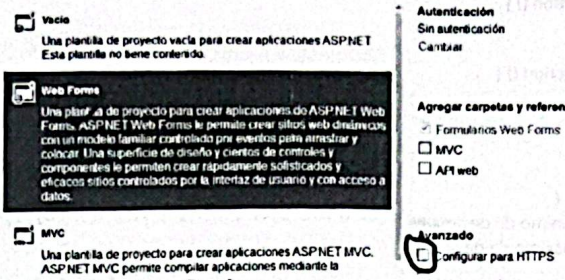
Actividad 4. Aplicación para entorno WEB para realizar la captura, el uso del servicio y muestra de datos, utilizando el servicio en la Web API implementada anteriormente.

a. En Microsoft Visual Studio 2022, utilizar la opción: ***Nuevo Proyecto***, crear un proyecto tipo: *Aplicación web ASP.NET (.NET Framework)* en **C#**, con nombre: **webSusCopias**, en la carpeta creada en la actividad 3(a). *Crear.*

Te lleva a la ventana de *Crear una aplicación web ASP.NET*, en ella seleccionar: ***Web Forms*** y deschequear la opción: **configurar para HTTPS**, luego: *Crear*

Powered by CamScanner

Crear una aplicación web ASP.NET



b. Al proyecto agregar una carpeta con nombre: ***Paginas***
c. En la carpeta anterior, agregar una página **HTML** con nombre: ***frmFact.***
c. Establecerla como página de inicio.
d. En ***frmFact*** diseñar los objetos en HTML de entrada y salida propuestos acorde a la siguiente imagen, utilizando 3 tablas centradas con ancho del 80%. **Nota**: Se utilizará JavaScript.

Facturación Fotocopias

Datos Facturación

Tipo Copia	Vr. Unit.	Cantidad	Valor
Carta	100		
Oficio	200		
ExtraOficio	300		

Limpiar | Procesar

Datos de Cobro

		SubTotal:	
% Dscto.:		Vr. Dscto:	
		Vr. IVA:	
		Valor a Pagar:	

Utilizar propiedades de tabla: **width, border, align y colspan.**

(Title): Práctica # 3 – API WEB
Tabla de 2 x 1, sin borde, 70%, centrada
En 1ra Línea: (Text): Facturación Fotocopias
En 2da Línea: (Text): Datos Facturación
Tabla de 8 x 4, con borde = 2, 90%, centrada
En 2da Línea:
Títulos: Tipo Copia, Vr. Unit., Cantidad, Valor
Todo este bloque con class="txt-cent"
En 3ra Línea:
Input (Text), id = **txtVrCarta**, disabled
Input (Text), id = **txtCantCarta**
Input (Text), id = **txtSubTotCarta**, disabled
En 4ta Línea:
Input (Text), id = **txtVrOfic**, disabled
Input (Text), id = **txtCantOfic**
Input (Text), id = **txtSubTotOfic**, disabled
En 5ta Línea:
Input (Text), id = **txtVrExtOfic**, disabled
Input (Text), id = **txtCantExtOfic**
Input (Text), id = **txtSubTotExtOfic**, disabled
En 7ma Línea: utilizar colspan
Input (Button), id=: **btnLimpiar**
Input (Button), id=: **btnProcesar**
Tabla de 5 x 4, con borde = 2, 70%, centrada
En 1ra Línea: (Text): Datos Cobro, utilizar **colspan**
Todo este bloque con class="txt-dere"
En 2da Línea
Input (Text), id = **txtSubTot**, readonly
En 3ra Línea
Input (Text), id = **txtPorcDsc**, readonly
Input (Text), id = **txtDscto**, readonly
En 4ta Línea:
Input (Text), id = **txtIva**, readonly
En 5ta Línea:
Input (Text), id = **txtAPagar**, readonly

e. Adicionar el siguiente script para textos en la página:

```
<style>
  .txt-dere {
    text-align: right;
  }
  .txt-cent {
    text-align:center;
  }
  .txt-titulo {
    font-family: 'Comic Sans MS';
    font-size: 20px;
    color: #9C2007;
    line-height: 1.5;
    text-align: center;
    font-style: italic;
    font-weight: bold;
  }
  .txt-subtit {
    font-family: cursive;
    font-size: 16px;
    color: #291980;
    text-align: center;
    font-style: italic;
    font-weight: bold;
  }
</style>
```

Actividad 5. Integración entre el formulario y el servicio en la API

Para la integración se utilizará código JavaScript y la estructura de los archivos JSON para el request y el response entre el cliente y el servidor del Web Server.

a. Crear la carpeta **scripts_form**, agregarle un archivo de tipo **JavaScript** con nombre: **ctlFrmFact**; crear las funciones de control de los objetos tipo: Button para el evento clic de cada uno.

var dir = "http://localhost:XXXXX/api/"; //Dirección servidor - Cambiar las XXXXX por el nro. del puerto del servidor API

Powered by CamScanner

```
jQuery( function ( ) {
//   pedirValores( );
   $("#btnLimpiar").on("click", function () {
      alert("Limpiar");
   } );
   $("#btnProcesar").on("click", function () {
      alert("Procesar");
   } );
} );
function fnro( vr ) {
   return vr.toLocaleString( 'es-CO', {
     minimumFractionDigits: 2, // Mínimo de decimales
     maximumFractionDigits: 2  // Máximo de decimales
   } );
}
function validarVr( valor, txt ) {
   if ( isNaN( valor ) || parseFloat( valor, 10 ) <= 0 || valor.trim( ) == "" ) {
      alert( "Error, valor del tipo: " + txt + ", no es válido" );
      return true;
   }
   else {
     return false;
   }
}
function pedirValores( ) {
   let vrCarta = prompt( "¿Cuál es el valor de cada copia: Carta:?", "" );
   let vrOfici = prompt( "¿Cuál es el valor de cada copia: Oficio:?", "" );
   let vrExtOf = prompt( "¿Cuál es el valor de cada copia: ExtraOficio:?", "" );
   // Verificamos si el usuario ingresó un valor
   if ( validarVr( vrCarta, "Carta" ) || validarVr( vrOfici, "Oficio" ) || validarVr( vrExtOf, "ExtraOficio" ) ) {
      return pedirValores();
   }
   else {
      $("#txtVrCarta").val( vrCarta );
      $("#txtVrOfic").val( vrOfici );
      $("#txtVrExtOfic").val( vrExtOf );
   }
}
function limpiarTotal( ) {
   $("#txtCantCarta").val("");
   $("#txtCantOfic").val("");
   $("#txtCantExtOfic").val("");
   limpiarRpta( );
}
function limpiarRpta( ) {
   $("#txtSubTotCarta").val("");
   $("#txtSubTotOfic").val("");
   $("#txtSubTotExtOfic").val("");
   $("#txtSubTot").val("");
   $("#txtPorcDsc").val("");
   $("#txtDscto").val("");
   $("#txtIva").val("");
   $("#txtAPagar").val("");
   $("#txtCantCarta").trigger("focus");
}
//La función tiene que ser asíncrona para que pueda funcionar el fetch al invocar el servicio
async function Procesar( ) {  //Definir vbles de entrada  -  let: vble local,  var vble global
   let vrCa = $("#txtVrCarta").val( );
   let vrOf = $("#txtVrOfic").val( );
   let vrEx = $("#txtVrExtOfic").val( );
   let kCa = $("#txtCantCarta").val( );
   let kOf = $("#txtCantOfic").val( );
   let kEx = $("#txtCantExtOfic").val( );
   //Validar
   if ( vrCa == undefined || isNaN( vrCa ) ) {
```


Powered by CamScanner

```
      alert( "Error, valor tipo carta, no válido" );
      $("#txtVrCarta").trigger("focus");
      return;
    }
    if ( vrOf == undefined || isNaN( vrOf ) ) {
      alert( "Error, valor tipo oficio, no válido" );
      $("#txtVrOfic").trigger( "focus" );
      return;
    }
    if ( vrEx == undefined || isNaN( vrEx ) ) {
      alert( "Error, valor tipo extraOficio, no válido" );
      $("#txtVrExtOfic").trigger( "focus" );
      return;
    }
    limpiarRpta( );
    //Crear la estructura de Json, con los datos de entrada deben coincidir con los nombres de las propiedades del modelo
    let datosOut = {
      vrC: vrCa,
      vrO: vrOf,
      vrE: vrEx,
      kC: kCa,
      kO: kOf,
      kE: kEx
    };
    //Invocar el servicio  -  Enviar info y recuperar respuestas
    try {  //Utilizar el método: fetch de JavaScript (Asíncrono) que conecta con la API con path de donde corre el servicio
      const retornoSrv = await fetch( dir + "servSusCopias",      // fetch: realiza(envía) peticiones de red asíncronas hacia un servidor o API
        {  method: "POST",                                        // proporciona una forma fácil y lógica de obtener recursos de forma asíncrona por la red
          mode: "cors",
          headers: {
            "Content-Type": "application/json"
          },
          body: JSON.stringify( datosOut ),
        }  //JSON.stringify:
      ); //Convierte el texto retornoSrv, que está como un texto JavaScript, en un texto de formato Json
      //Recuperar y leer la respuesta del servicio
      const Rpta = await retornoSrv.json( );     //En la constante Rpta, queda la respuesta del servicio, en formato: JSon
      let error = Rpta.Error;
      if ( error != "" ) {
        alert( "Error, " + error );
        return;
      }
      $("#txtSubTotCarta").val( fnro(Rpta.vrTotC) );
      $("#txtSubTotOfic").val( fnro(Rpta.vrTotO) );
      $("#txtSubTotExtOfic").val( fnro(Rpta.vrTotE) );
      $("#txtSubTot").val( fnro(Rpta.vrSubTot) );
      $("#txtPorcDsc").val( fnro(Rpta.porcDscto) );
      $("#txtDscto").val( fnro(Rpta.vrDscto) );
      $("#txtIva").val( fnro(Rpta.vrIva) );
      $("#txtAPagar").val( fnro(Rpta.vrAPag) );
    }
    catch ( e ) {
      alert( "Error, " + e );
    }
  }
```

b. Desde la carpeta: Scripts, adicionar los scripts: **jquery-3.7.0.min** a la página HTML al final del body.
c. Desde la carpeta: scripts_form, adicionar el script: **ctlFrmFact**
d. Probar con los siguientes datos:

Tipo	Vr. Unit	Cantidad	vr_PorTipo
Carta	100	400	40000
Oficio	200	300	60000
ExtraOfic	300	100	30000

SubTotal		130.000,00
Dscto	15%	19.500,00
IVA		8.287,50
APAgar		118.787,50

Powered by CamScanner

Tema de estudio

¿Qué es una API?

Las API son mecanismos que permiten a dos componentes de software comunicarse entre sí mediante un conjunto de definiciones y protocolos. Por ejemplo, el sistema de software del instituto de meteorología contiene datos meteorológicos diarios. La aplicación meteorológica de su teléfono "habla" con este sistema a través de las API y le muestra las actualizaciones meteorológicas diarias en su teléfono.

¿Qué significa API?

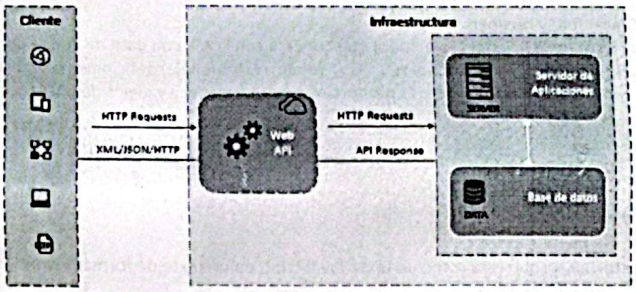
API significa "interfaz de programación de aplicaciones". En el contexto de las API, la palabra aplicación se refiere a cualquier software con una función distinta. La interfaz puede considerarse como un contrato de servicio entre dos aplicaciones. Este contrato define cómo se comunican entre sí mediante solicitudes y respuestas. La documentación de su API contiene información sobre cómo los desarrolladores deben estructurar esas solicitudes y respuestas.

¿Cómo funcionan las API?

La arquitectura de las API suele explicarse en términos de cliente y servidor. La aplicación que envía la solicitud se llama cliente, y la que envía la respuesta se llama servidor. En el ejemplo del tiempo, la base de datos meteorológicos del instituto es el servidor y la aplicación móvil es el cliente.

Arquitectura básica de una Web API

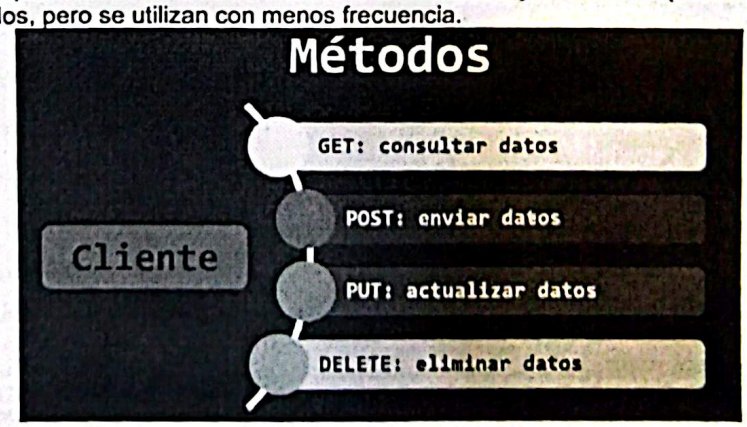
Arquitectura Web API



Nota: esta arquitectura está basado para On-Premise (en local), quiere decir que los servidores de aplicación y base de datos estarán dentro de la infraestructura de la organización (no cloud computing).

Métodos

Los métodos HTTP principales o más utilizados son: POST, GET, PUT y DELETE. Estos métodos corresponden a las operaciones de lectura, creación, actualización y eliminación (o CRUD), respectivamente. Existen otros métodos, pero se utilizan con menos frecuencia.



- **GET:** Se utiliza para recuperar un recurso
- **POST:** Se utiliza para crear un recurso nuevo o enviar datos a un recurso existente
- **PUT:** Se utiliza para modificar un recurso existente
- **PATCH:** Se utiliza para modificar solamente un atributo de un recurso
- **DELETE:** Se utiliza para eliminar un recurso determinado

Powered by CamScanner

API	Descripción	Cuerpo de la solicitud	Cuerpo de respuesta
GET /api/APIActiv	Obtén todos los elementos de la lista de tareas pendientes.	Ninguno	Conjunto de tareas pendientes
GET /api/APIActiv /{id}	Obtener un artículo por ID	Ninguno	Elemento de la lista de tareas pendientes
POST /api/APIActiv	Agregar un nuevo artículo	Elemento de la lista de tareas pendientes	Elemento de la lista de tareas pendientes
PUT /api/APIActiv /{id}	Actualizar un elemento existente	Elemento de la lista de tareas pendientes	Ninguno
DELETE /api/APIActiv /{id}	Eliminar un elemento	Ninguno	Ninguno

Estos métodos se alinean con las operaciones CRUD (Crear, Leer, Actualizar y Eliminar), que se utilizan para manipular recursos en la web.

USO

Las API pueden funcionar de cuatro maneras diferentes, según el momento y el motivo de su creación.

API de SOAP

Estas API utilizan el protocolo simple de acceso a objetos. El cliente y el servidor intercambian mensajes mediante XML. Se trata de una API menos flexible que era más popular en el pasado.

API de RPC

Estas API se denominan llamadas a procedimientos remotos. El cliente completa una función (o procedimiento) en el servidor, y el servidor devuelve el resultado al cliente.

API de WebSocket

La API de WebSocket es otro desarrollo moderno de la API web que utiliza objetos JSON para transmitir datos. La API de WebSocket admite la comunicación bidireccional entre las aplicaciones cliente y el servidor. El servidor puede enviar mensajes de devolución de llamada a los clientes conectados, por lo que es más eficiente que la API de REST.

API de REST

Estas son las API más populares y flexibles que se encuentran en la web actualmente. El cliente envía las solicitudes al servidor como datos. El servidor utiliza esta entrada del cliente para iniciar funciones internas y devuelve los datos de salida al cliente. Veamos las API de REST con más detalle a continuación.

¿Qué son las API de REST?

REST significa transferencia de estado representacional. REST define un conjunto de funciones como GET, PUT, DELETE, etc. que los clientes pueden utilizar para acceder a los datos del servidor. Los clientes y los servidores intercambian datos mediante HTTP.

La principal característica de la API de REST es que no tiene estado. La ausencia de estado significa que los servidores no guardan los datos del cliente entre las solicitudes. Las solicitudes de los clientes al servidor son similares a las URL que se escriben en el navegador para visitar un sitio web. La respuesta del servidor son datos simples, sin la típica representación gráfica de una página web.

¿Qué es una API web?

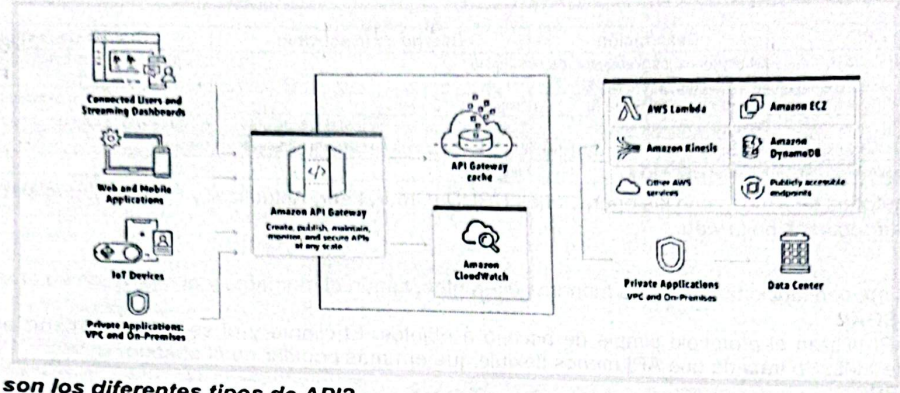
Una API web o API de servicios web es una interfaz de procesamiento de aplicaciones entre un servidor web y un navegador web. Todos los servicios web son API, pero no todas las API son servicios web. La API de REST es un tipo especial de API web que utiliza el estilo arquitectónico estándar explicado anteriormente.

Los diferentes términos relacionados con las API, como API de Java o API de servicios, existen porque históricamente las API se crearon antes que la World Wide Web. Las API web modernas son API de REST y los términos pueden utilizarse indistintamente.

¿Qué son las integraciones de las API?

Las integraciones de las API son componentes de software que actualizan automáticamente los datos entre los clientes y los servidores. Algunos ejemplos de integraciones de las API son la sincronización automática de datos en la nube desde la galería de imágenes de su teléfono o la sincronización automática de la hora y la fecha en su laptop cuando viaja a otra zona horaria. Las empresas también pueden utilizarlas para automatizar de manera eficiente muchas funciones del sistema.

Powered by CamScanner



¿Cuáles son los diferentes tipos de API?

Las API se clasifican tanto en función de su arquitectura como de su ámbito de uso. Ya exploramos los principales tipos de arquitecturas de API, ahora veamos el ámbito de uso.

API privadas

Estas son internas de una empresa y solo se utilizan para conectar sistemas y datos dentro de la empresa.

API públicas

Están abiertas al público y pueden cualquier persona puede utilizarlas. Puede haber o no alguna autorización y coste asociado a este tipo de API.

API de socios

Solo pueden acceder a ellas los desarrolladores externos autorizados para ayudar a las asociaciones entre empresas.

API compuestas

Estas combinan dos o más API diferentes para abordar requisitos o comportamientos complejos del sistema.

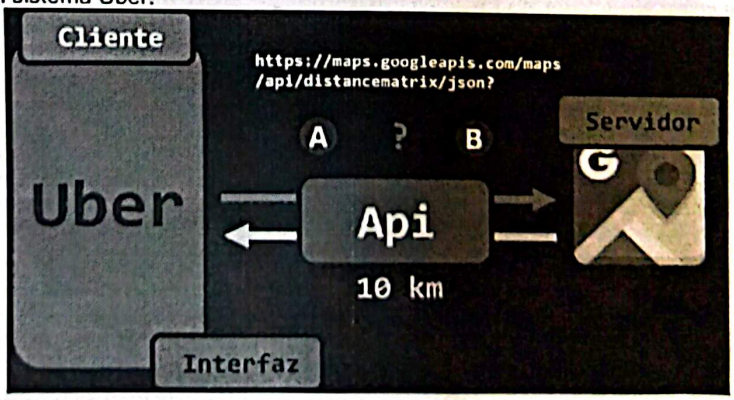
Ejemplos

https://aws.amazon.com/es/what-is/api/

Por ejemplo, piensa que quieres crear una aplicación que se conecte a las publicaciones que hay en Twitter, pues para ello tendrás que conectar tu aplicación al servicio mediante la API que Twitter tiene disponible para los desarrolladores. Y si quieres que esta aplicación se comunique con la barra de notificaciones de un sistema operativo, pues también necesitará otra API del sistema operativo.

Piensa si no en cuando vas a comprar una entrada a través de la web de una sala de cine. Cuando pones la información de tu tarjeta, la web utiliza una API para **enviarle esa información de forma remota a otro programa** que verifica si tus datos son correctos o es una tarjeta inventada. Una vez se verifica, este programa remoto le dice a la web que todo está en orden, y está ya te emite tus entradas.

Otro ejemplo es el sistema Uber.



Otro ejemplo clásico es el de las aplicaciones de terceros para conectarse a un servicio. Por ejemplo, redes sociales como Twitter o Reddit permiten que se creen **aplicaciones diferentes a las oficiales para conectarse a ellas**. Estas aplicaciones necesitarán las API de las redes sociales para poder mostrarte la información y los mecanismos internos que las hacen funcionar.

En estos casos, las API le pueden permitir a estos servicios o redes sociales limitar qué funciones quieren que las aplicaciones de terceros puedan utilizar. Por ejemplo, muchas veces se ha informado de que Twitter les pone trabas a estos desarrolladores haciendo que las API limiten el número de usuarios que las puedan utilizar. A veces otros servicios crean API de forma deliberada para ser utilizados por terceros en tareas concretas, y así **extender su uso y popularidad** creando nuevas funciones. Por ejemplo, Google creó una para Google Docs con las que permite la creación automatizada de facturas o informes de ventas a otros servicios.

Y, para terminar, otro ejemplo. Piensa en todos esos servicios que funcionan online teniendo la información en la nube. Pues para poder hacer eso no hace falta que se monten sus propios servidores, sino utilizar **las API de los servicios en la nube** de empresas como Google, Amazon o Microsoft para poder utilizar los suyos.

Otros 3 ejemplo de API son:

1. API de plataformas de marketing
   Las API son un medio utilizado en distintas plataformas de marketing, como las que se ofrecen en HubSpot, para robustecer las funciones con las que cuentan.
   Diversos desarrolladores colaboran para abrir funciones muy específicas. Como APIs para:
   - Análisis y reportes
   - Automatizar flujos de trabajo
   - Plantillas de blog
   - Redirecciones
   - Funciones de chat extendidas
   - Búsquedas en el CRM
   - Indicadores de email marketing, y mucho más
2. API de menciones de Twitter
   Otro ejemplo son las API de menciones. Por ejemplo, podrías contar con una interfaz que muestre los tuits que llevan a tu sitio web o blog, siempre y cuando la API pueda identificar las menciones de una URL específica.
   De esta forma, las personas que visiten tu sitio web sabrán que estás generando una conversación interesante con tus seguidores y, por ende, tu marca es experta en su industria.
3. API de incrustación de videos de YouTube
   Cuando haces clic con el botón secundario en un video de YouTube en youtube.com, selecciona la opción de copiar el código de inserción, con el que estarás solicitando usar la API de YouTube en tu sitio web. Esta plataforma facilita insertar videos propios para que puedan reproducirse directamente en otros sitios web.
   El desarrollo de las API de servicios web es solo el comienzo. En el mundo actual, se espera que haya contenido abierto y disponible para todos. La evolución natural de esto es que los sitios web publiquen sus propias API de modo que sus clientes puedan desarrollar aplicaciones como resultado.
   El uso compartido de las API se aplica a todas las empresas y no solo a aquellas que están basadas en la web. Cualquiera que tenga una herramienta o un componente informático puede hacer uso de estas herramientas. Depende de ti descubrir cuáles son las API más valiosas. Úsalas adecuada y legalmente para que tu negocio crezca.

https://blog.hubspot.es/website/que-como-usar-api

Qué es Cors

Cross-Origin Resource Sharing (CORS) en inglés o también: entre orígenes o cruzado (CORS) es una característica de seguridad del navegador que restringe las solicitudes HTTP de origen cruzado que se inician desde secuencias de comandos que se ejecutan en el navegador.

Determinar si se habilita la compatibilidad con CORS
Una solicitud HTTP *de origen cruzado* es una que se hace para:

- Un *dominio* diferente (por ejemplo, de example.com a amazondomains.com)
- Un *subdominio* diferente (por ejemplo, de example.com a petstore.example.com)
- Un *puerto* diferente (por ejemplo, de example.com a example.com:10777)
- Un *protocolo* diferente (por ejemplo, de https://example.com a http://example.com)

Powered by CamScanner

https://docs.aws.amazon.com/es_es/apigateway/latest/developerguide/how-to-cors.html

For security reasons, browsers restrict cross-origin HTTP requests initiated from scripts. For example, XMLHttpRequest and the Fetch API follow the same-origin policy. This means that a web application using those APIs can only request resources from the same origin the application was loaded from unless the response from other origins includes the right CORS headers.

Origen de uso de la API

Dos URL tienen el mismo origen si tienen esquemas, hosts y puertos idénticos (RFC 6454).

Estas dos direcciones URL tienen el mismo origen:

- https://example.com/foo.html
- https://example.com/bar.html

Estas direcciones URL tienen orígenes diferentes a las dos direcciones URL anteriores:

- https://example.net: Dominio diferente
- https://contoso.example.com/foo.html: Subdominio diferente
- http://example.com/foo.html: Esquema diferente
- https://example.com:9000/foo.html: Puerto diferente

https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS
https://learn.microsoft.com/es-es/aspnet/core/security/cors?view=aspnetcore-9.0

5. PARÁMETROS PARA ELABORACIÓN DEL INFORME

NA.

6. BIBLIOGRAFÍA

- ✓ Deitel, Harvey M.; Deitel, Paul J. Cómo programar en C#. - 2. Ed., México: Pearson Educación, 2007, 1166 p. 1CD ROM, ISBN: 9702610567, (ITM) 005.133 D325
- ✓ Charte Ojeda, Francisco; Serrano Pérez, Jorge. Programación con Visual Studio .NET, Madrid: Anaya Multimedia, 2002, 654 p. + CD-ROM, ISBN: 8441513767, (ITM) 005.133 C486
- ✓ Kingsley-Hughes, Adrian; Kingsley-Hughes, Kathie. C# 2005: Aprendizaje y referencia, Madrid: Anaya Multimedia, 2007, 448 p, ISBN: 9788441522381, (ITM) 005.133 K55
- ✓ Bradley, Julia Case; Millspaugh, Anita C. Programming in C# .NET, New York: McGraw-Hill, 2003, 556p. + CD-ROM, ISBN: 0071215646, (ITM) 005.133 B811
- ✓ Wright, Charles. Superutilidades para C#, Madrid: McGraw-Hill, 2003, 639p, ISBN: 9789701047834, (ITM) 005.133 W948
- ✓ Ramírez, Felipe. Introducción a la programación: algoritmos y su implementación en VB.NET, C#, Java y C++. - 2. Ed, México: Alfaomega, 2007, 488 p, ISBN: 9789701512807, (ITM) 005.1 R173

Elaborado por:	*Delio Augusto Aristizábal Martínez*
Versión:	*8*
Fecha:	*Agosto de 2026*
Aprobado por:	

Powered by CamScanner